



Université Alioune Diop
L'Excellence est ma constance, l'Éthique ma vertu !

RAPPORT DE PROJET

Analyse de Sentiment en NLP Comparaison TF-IDF vs DistilBERT

Master 2 - Data Science & Génie Logiciel
Année universitaire 2024-2025
Vision Senegal 2050

Portfolio : <https://pa-malick.github.io/portfolio/>

Table des matieres

1. Introduction et contexte
2. Presentation du dataset IMDB
3. Fondements theoriques du NLP et de l'analyse de sentiment
4. Architecture du projet
5. Preprocessing du texte
6. Modeles implementes
7. Entrainement et configuration
8. Evaluation et comparaison des resultats
9. Interface web Flask
10. Deploiement avec Docker
11. Deploiement Cloud (Render / Railway / Heroku)
12. Difficultes rencontrees et solutions
13. Perspectives d'amelioration
14. Conclusion
15. References bibliographiques

1. Introduction et contexte

L'analyse de sentiment est l'une des tâches les plus étudiées en Traitement du Langage Naturel (NLP). Elle consiste à déterminer automatiquement l'opinion exprimée dans un texte : positive, négative, ou neutre. Cette capacité est aujourd'hui au cœur de nombreuses applications industrielles : monitoring des réseaux sociaux, analyse des avis clients, veille concurrentielle, et bien d'autres.

Dans ce projet, nous avons construit un pipeline complet d'analyse de sentiment en nous appuyant sur le dataset de référence IMDB, constitué de 50 000 critiques de films. L'objectif était double : d'un côté construire des modèles performants, de l'autre développer une interface web moderne permettant de tester ces modèles en temps réel.

Nous avons comparé trois approches représentant l'évolution historique du NLP :

- Les approches classiques basées sur TF-IDF (fin des années 2000 - 2010s)
- Les modèles d'ensemble comme Random Forest
- Les architectures Transformer (DistilBERT) qui dominent l'état de l'art depuis 2018

Ce projet s'inscrit aussi dans la vision Senegal 2050 : former des ingénieurs en IA capables de contribuer à la transformation numérique du continent africain avec des technologies modernes et maîtrisées localement.

2. Présentation du dataset IMDB

Le Large Movie Review Dataset (IMDB) est l'un des benchmarks les plus utilisés en analyse de sentiment. Il a été introduit par Maas et al. en 2011 et contient 50 000 critiques de films en anglais, séparées en deux classes : positif (label=1) et négatif (label=0).

2.1 Caractéristiques principales

Caractéristique	Valeur
Total exemples	50 000
Exemples train	25 000
Exemples test	25 000
Distribution	50% positif / 50% négatif
Langue	Anglais
Longueur moyenne	~231 mots par critique

Le dataset est parfaitement équilibré, ce qui est idéal : on n'a pas besoin de techniques de rééquilibrage (SMOTE, sous-échantillonnage) qui complexifieraient le pipeline. On peut utiliser l'accuracy comme métrique principale sans risque de biais.

On charge le dataset via la librairie HuggingFace Datasets avec une seule ligne de code : `load_dataset('imdb')`. Un système de cache local a été mis en place pour ne pas télécharger les données à chaque exécution.

3. Fondements theoriques - NLP et Analyse de Sentiment

3.1 Qu'est-ce que le NLP ?

Le Traitement du Langage Naturel (NLP) est la branche de l'IA qui permet aux machines de comprendre, interpreter et generer du langage humain. C'est un domaine interdisciplinaire qui combine linguistique, statistiques et apprentissage automatique.

3.2 TF-IDF - Term Frequency Inverse Document Frequency

TF-IDF est une technique de vectorisation classique qui transforme un texte en vecteur numerique. L'idee est simple : un mot est important dans un document s'il y apparait souvent (TF elevee) mais pas dans trop de documents du corpus (IDF elevee).

Formule :

$TF(t, d) = \text{nombre occurrences du terme } t \text{ dans le document } d / \text{nombre total de termes dans } d$

$IDF(t) = \log(N / df(t))$ ou $N = \text{nombre de documents}$, $df(t) = \text{nombre de documents contenant } t$

$TF-IDF(t, d) = TF(t, d) * IDF(t)$

Dans notre projet, on utilise des bigrammes ($ngram_range=(1,2)$) pour capturer des expressions comme 'not good' ou 'very bad'. Sans bigrammes, 'not good' serait interprete comme deux mots separes, perdant toute la nuance de la negation.

3.3 Regression Logistique

La Regression Logistique est un classifieur lineaire qui estime la probabilite qu'un exemple appartienne a une classe. Malgre son nom, c'est bien un algorithme de classification. Elle utilise la fonction sigmoide pour projeter les scores lineaires dans $[0, 1]$.

Dans notre contexte, chaque critique est representee par un vecteur TF-IDF de 30 000 dimensions. Le modele apprend un poids pour chaque dimension (chaque mot ou bigramme) indiquant s'il est plutot associe a un sentiment positif ou negatif.

Avantage important : la Regression Logistique est interpretable. On peut regarder les poids les plus eleves pour comprendre quels mots contribuent le plus a la prediction.

3.4 Random Forest

Le Random Forest est un modele d'ensemble qui combine plusieurs arbres de decision. Chaque arbre est entraine sur un sous-echantillon aleatoire des donnees et des features, puis la prediction finale est le vote majoritaire de tous les arbres.

En NLP avec TF-IDF, le Random Forest est generalement moins performant que la Regression Logistique. Pourquoi ? Parce que la representation TF-IDF cree un espace de tres haute dimension (30 000+), et les arbres de decision ont du mal avec autant de features. C'est un resultat contre-intuitif pour les debutants, mais tres educatif.

3.5 DistilBERT - Transformer allégée

BERT (Bidirectional Encoder Representations from Transformers) a révolutionné le NLP en 2018 (Devlin et al., 2018). Il pré-entraîne un grand modèle sur de vastes corpus textuels en utilisant deux tâches : Masked Language Modeling et Next Sentence Prediction.

DistilBERT (Sanh et al., 2019) est une version distillée de BERT : 40% moins de paramètres, 60% plus rapide, mais 97% des performances de BERT. La distillation de connaissances est une technique où un petit modèle (étudiant) apprend en imitant un grand modèle (professeur).

Nous fine-tunons DistilBERT sur IMDB : on part des poids pré-entraînés et on ajuste tous les paramètres sur notre tâche spécifique. C'est cette approche de transfert d'apprentissage qui explique les excellentes performances avec relativement peu de données.

3.6 Métriques d'évaluation

Métrique	Formule	Ce qu'elle mesure
Accuracy	$TP + TN / \text{Total}$	Taux global de bonnes prédictions
Precision	$TP / (TP + FP)$	Parmi les positifs prédits, combien sont vrais
Recall	$TP / (TP + FN)$	Parmi les vrais positifs, combien sont détectés
F1-Score	$2 * P * R / (P + R)$	Équilibre entre Precision et Recall

4. Architecture du projet

Le projet est structuré de façon modulaire pour séparer clairement les responsabilités. Cette organisation facilite la maintenance, les tests et l'extension future.

4.1 Organisation des modules

Fichier	Rôle
data_loader.py	Charge et met en cache le dataset IMDB depuis HuggingFace
preprocessing.py	Nettoie les textes et construit le vectoriseur TF-IDF
models.py	Définit les trois classes de modèles (LR, RF, DistilBERT)
train.py	Fonctions d'entraînement, Dataset PyTorch pour BERT
evaluation.py	Calcul des métriques, génération des graphiques
utils.py	Fonctions partagées, point d'entrée pour l'API Flask

4.2 Flux de traitement

- 1. Chargement : `load_dataset('imdb')` -> 50 000 critiques avec labels
- 2. Preprocessing : nettoyage HTML, stopwords, ponctuation -> texte propre
- 3. Vectorisation : TF-IDF (classiques) ou tokenisation BERT
- 4. Entrainement : fit sklearn ou boucle PyTorch avec AdamW
- 5. Evaluation : predictions sur le test set, calcul des metriques
- 6. Sauvegarde : joblib pour sklearn, `save_pretrained` pour BERT
- 7. Interface web : Flask charge les modeles et expose une API REST

5. Preprocessing du texte

Le preprocessing est une etape cruciale en NLP. Des donnees mal nettoyees degradent significativement les performances, surtout pour les modeles classiques qui ne comprennent pas le contexte.

5.1 Etapes de nettoyage (modeles classiques)

- Mise en minuscule : 'Great' et 'great' doivent etre traitees de la meme facon
- Suppression des balises HTML : les critiques IMDB contiennent parfois des `<br>` ou `<p>`
- Suppression de la ponctuation et des chiffres : ils apportent peu d'information semantique
- Suppression des stopwords : 'the', 'is', 'a', 'an' n'ont pas de valeur discriminante
- Suppression des tokens trop courts (longueur ≤ 2) : souvent du bruit

5.2 Pourquoi on ne supprime pas les stopwords pour BERT

Pour DistilBERT, on ne fait pas ce nettoyage agressif. BERT comprend le langage en contexte : la phrase 'not good at all' a un sens tres different de 'good'. Si on supprime 'not' et 'at', on perd toute la negation. BERT capture ces subtilites grace a l'attention bidirectionnelle, mais seulement si les mots sont presents.

5.3 Choix des bigrammes

Avec `ngram_range=(1,2)`, le vectoriseur TF-IDF considere non seulement les mots individuels mais aussi les paires de mots consecutifs. Exemples de bigrammes utiles : 'not good', 'very bad', 'must watch', 'waste time'. Cela permet de capturer une partie du contexte local sans les Transformers.

6. Modeles implementes

6.1 TfidfLogisticModel

Parametres choisis :

`C=1.0` : regularisation L2 par default, bon compromis biais-variance

`max_iter=1000` : convergence garantie sur des datasets denses

`solver='lbfgs'` : efficace pour les problemes de grande dimension

`n_jobs=-1` : utilise tous les coeurs disponibles

6.2 TfidfRandomForestModel

Parametres choisis :

`n_estimators=200` : 200 arbres pour une prediction stable

`random_state=42` : reproductibilite des resultats

`n_jobs=-1` : parallelisation sur tous les coeurs

Note : 200 arbres sur 30 000 features TF-IDF prend quelques minutes. Pour aller plus vite lors des tests, on peut reduire a 50 estimateurs.

6.3 DistilBERTClassifier

Architecture :

On charge `distilbert-base-uncased` depuis HuggingFace et on ajoute une couche de classification lineaire au-dessus (`DistilBertForSequenceClassification`). Cette couche finale prend en entree la representation [CLS] du dernier layer et predit les logits pour les deux classes.

Hyperparametres de fine-tuning :

Hyperparametre	Valeur	Justification
Learning rate	2e-5	Standard pour le fine-tuning BERT
Epochs	3	Suffisant, plus risque de surapprentissage
Batch size	16	Compromis vitesse/memoire GPU
Max length	256	Les critiques IMDB sont rarement plus longues
Warmup steps	10% du total	Stabilise le debut de l'entrainement
Weight decay	0.01	Regularisation AdamW standard

7. Entrainement et configuration

7.1 Pipeline d'entrainement classique

Pour les modeles classiques, le pipeline est simple et rapide. Preprocessing -> TF-IDF -> `sklearn fit()`. L'ensemble de l'entrainement prend moins de 5 minutes sur un CPU standard.

7.2 Pipeline DistilBERT

Pour DistilBERT, on utilise un Dataset PyTorch personnalis   (IMDBDataset) qui tokenise les textes a la vol  e. Le DataLoader gere le batching et le shuffling. On utilise AdamW avec un scheduler lineaire qui reduit progressivement le learning rate.

Astuce importante : on clippe les gradients a `max_norm=1.0`. Sans ca, le fine-tuning peut   tre instable dans les premiers batches. C'est une pratique standard pour les Transformers.

7.3 Mode sample pour les tests rapides

On a implement   un flag `--sample` qui reduit le dataset a 5 000 exemples train et 1 000 exemples test. Cela permet de tester tout le pipeline en quelques minutes sans attendre un entra  nement complet. Tres utile pendant le developpement.

8. Evaluation et resultats

8.1 Resultats obtenus

Modele	Accuracy	F1-Score	Temps entra��nement
Logistic Regression	88.40%	88.40%	< 2 min (CPU)
Random Forest	87.20%	87.20%	5-10 min (CPU)
DistilBERT fine-tune	89.20%	89.20%	��85 min (CPU)

8.2 Analyse des resultats

Sur notre jeu de test   quilibre (1000 exemples, 500 negatifs + 500 positifs), la Regression Logistique atteint 88.40% d'accuracy, le Random Forest 87.20%, et DistilBERT fine-tune 89.20%. L'ecart entre LR et DistilBERT est de seulement 0.8 points de pourcentage, ce qui est caracteristique d'une t  che binaire bien d  finie avec un dataset propre et   quilibre.

La Regression Logistique est surprenante : presque aussi bonne que DistilBERT avec un temps d'entra  nement des centaines de fois inferieur. C'est un resultat classique en NLP : sur des t  ches binaires bien d  finies avec beaucoup de donnees, les modeles lineaires sont tres competitifs.

Le Random Forest est en dessous de la Regression Logistique. Cela s'explique par la nature de l'espace TF-IDF : tres creux (sparse) et de haute dimension. Les arbres de decision ne sont pas optimaux dans cet espace.

DistilBERT gagne sur la confiance : ses predictions atteignent 98%+ de certitude contre 72-83% pour les modeles classiques. Il capture la negation, l'ironie et les expressions idiomatiques que TF-IDF ne peut pas représenter. Sur le dataset complet (25 000 exemples), l'ecart serait plus marque.

8.3 Limites de l'évaluation

Notre évaluation est sur IMDB uniquement (anglais, critiques de films). Les performances pourraient être différentes sur d'autres domaines (e-commerce, santé) ou d'autres langues (français, wolof). La généralisation n'est pas garantie.

9. Interface web Flask

L'interface web est construite avec Flask, un microframework Python léger et flexible. On a choisi Flask plutôt que Django car le projet ne nécessite pas de base de données ni d'ORM : Flask suffit largement pour une API REST simple.

9.1 Architecture Flask

On utilise le pattern Blueprint pour séparer les routes de la logique d'initialisation. La fonction `create_app()` crée et configure l'application, ce qui facilite les tests (on peut créer une app de test séparée).

9.2 Design futuriste Senegal 2050

L'interface utilise une palette inspirée du Sénégal : vert #00853F, or #F0B429, rouge #E31B23 du drapeau sénégalais, sur un fond sombre futuriste. Le design utilise le glassmorphism (effet verre translucide) et des animations GSAP pour les transitions.

GSAP (GreenSock Animation Platform) est la librairie de référence pour les animations web. On l'utilise pour : l'entrée de la navbar, le titre hero, le scroll reveal des cartes, et les particules flottantes en arrière-plan.

9.3 Endpoints API

Methode	Route	Description
GET	/	Page d'accueil avec le formulaire
GET	/about	Page de présentation du projet
POST	/predict	Prediction JSON : {text, model} -> {sentiment, confidence}
GET	/status	Etat des modèles disponibles
GET	/metrics	Dernières métriques d'évaluation

10. Déploiement avec Docker

Docker permet de packager toute l'application (code + dépendances + config) dans une image portable. Ça résout le problème classique 'ça marche sur ma machine'. Avec docker-compose, on peut lancer l'application avec une seule commande.

10.1 Structure Docker

On definit deux services dans docker-compose.yml :

sentiment-app : le serveur Flask en production (gunicorn)

sentiment-train : l'entrainement des modeles (profil optionnel)

Les dossiers models/ et metrics/ sont montes en volumes pour persister les modeles entraines et les graphiques entre les redemarrages du container.

10.2 Commandes utiles

docker-compose up -> Lance l'app web

docker-compose --profile train up sentiment-train -> Entraîne les modeles

docker build -t sentiment-ai . -> Construit l'image

11. Deploiement Cloud (Render / Railway / Heroku)

En complement du deploiement Docker local, le projet est pret a etre deploye sur des plateformes cloud gratuites ou a faible cout. Trois options ont ete configurees : Render, Railway et Heroku. Chacune lit le port a utiliser depuis la variable d'environnement PORT, injectee automatiquement par la plateforme.

11.1 Fichiers de configuration crees

Fichier	Plateforme	Role
render.yaml	Render.com	Deploiement Docker automatique depuis GitHub
railway.toml	Railway.app	Build Dockerfile, healthcheck /status
Procfile	Heroku / Railway	Commande gunicorn avec \$PORT dynamique
heroku.yml	Heroku (Docker)	Deploiement base Docker sur Heroku
.dockerignore	Toutes	Accelere le build en excluant les fichiers inutiles
.env.example	Toutes	Template des variables d'environnement necessaires

11.2 Deploiement sur Render (recommande)

- 1. Pousser le code sur GitHub (avec les modeles .pkl dans le repo ou via Git LFS)
- 2. Creer un compte sur render.com et connecter le depot GitHub
- 3. Render detecte automatiquement render.yaml et configure le service Docker
- 4. Le build installe les dependances et lance gunicorn sur le port fourni par Render
- 5. L URL publique est disponible immediatement apres le premier deploiement

11.3 Deploiement sur Railway

- 1. Creer un projet sur railway.app et connecter le depot GitHub
- 2. Railway detecte railway.toml et utilise le Dockerfile
- 3. La variable PORT est injectee automatiquement, gunicorn l ecoute
- 4. Le healthcheck sur /status verifie que l application est bien demarre

11.4 Deploiement sur Heroku

Heroku utilise heroku.yml pour un deploiement base sur Docker. La commande principale reste la meme : gunicorn avec \$PORT. Note : le plan gratuit Heroku a ete supprime en 2022, le plan Eco (5\$/mois) reste l option la plus abordable.

11.5 Note sur les modeles et la taille de l image

Le fichier .dockerignore exclut les caches et fichiers temporaires mais conserve les modeles entraines (dossier models/). L image Docker finale inclut donc les fichiers .pkl (TF-IDF + classifieurs) et le modele DistilBERT fine-tune. La taille totale de l image est d environ 2 a 3 Go. Pour les plans gratuits avec des limites de RAM (512 Mo sur Render free), il est conseille de deployer uniquement les modeles classiques (Logistic Regression, Random Forest) qui consomment nettement moins de memoire.

12. Difficultes rencontrees et solutions

Probleme : Memoire insuffisante pour DistilBERT en CPU

Solution : Implementer le flag --sample pour reduire la taille du dataset. Aussi, on charge les donnees en batches avec DataLoader au lieu de tout mettre en RAM.

Probleme : Cache du dataset IMDB

Solution : HuggingFace Datasets fait son propre cache, mais on a ajoute un cache pickle supplementaire dans models/ pour avoir un controle explicite et eviter les telechargements repetes.

Probleme : Gestion des modeles non entraines dans Flask

Solution : L'API retourne une erreur HTTP 503 claire si le modele n'est pas encore entraine, plutot que de crasher avec une erreur Python incomprehensible.

Probleme : Performance de Random Forest sur TF-IDF sparse

Solution : Contre-intuitif : RF est moins bon que LR sur des features TF-IDF. On a d'abord cru a un bug, mais c'est un comportement connu documente dans la litterature NLP.

Probleme : Animations GSAP et chargement asynchrone

Solution : Le resultat de prediction arrive de facon asynchrone (fetch API). On a separe les fichiers JS (main.js pour les animations globales, analyzer.js pour la logique de prediction) pour eviter les conflits.

13. Perspectives d'amelioration

- Support multilingue : etendre l'analyse au francais et au wolof avec des modeles multilingues (mBERT, XLM-R)
- Analyse fine des aspects : plutot que positif/negatif global, detecter le sentiment par aspect (jeu d'acteur, scenario, effets speciaux)
- API REST documentee avec Swagger/OpenAPI pour faciliter l'integration dans d'autres applications
- Dashboard de monitoring : visualiser en temps reel les predictions, les confiances, les textes les plus soumis
- Fine-tuning sur des domaines specifiques : sante, e-commerce, reseaux sociaux africains
- Quantification du modele BERT (INT8) pour un deploiement plus rapide sur des appareils limites
- Tests de robustesse : evaluer le comportement face a l'ironie, au sarcasme et aux fautes de frappe

14. Conclusion

Ce projet nous a permis de couvrir l'ensemble du cycle de vie d'un projet NLP : de la preparation des donnees jusqu'au deploiement d'une application web. Nous avons compare trois approches de complexite croissante et tire des conclusions interessantes sur les compromis performance/rapidite/interpretabilite.

La lecon principale est que les modeles classiques restent pertinents : la Regression Logistique avec TF-IDF atteint 89-91% d'accuracy sur IMDB, ce qui est excellent pour un modele entraine en moins de 2 minutes. DistilBERT gagne quelques points de pourcentage mais au prix d'une complexite et d'un temps d'entrainement bien plus importants.

D'un point de vue pratique, la Regression Logistique serait souvent recommandee en production pour des cas simples (binaire, texte anglais bien forme), tandis que DistilBERT s'impose quand on a besoin de comprendre des subtilites linguistiques complexes ou qu'on travaille sur des taches multiclassées.

Ce projet contribue modestement a la formation en IA au Senegal, en montrant qu'il est possible de construire des systemes NLP complets et performants avec des outils open-source accessibles.

15. References bibliographiques

[1] Maas, A. L., et al. (2011). Learning Word Vectors for Sentiment Analysis. Proceedings of the 49th Annual Meeting of the ACL, pp. 142-150.

[2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.

[3] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[5] Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of the 2020 Conference on EMNLP: System Demonstrations.

[6] Pang, B., & Lee, L. (2008). Opinion Mining and Sentiment Analysis. Foundations and Trends in Information Retrieval, 2(1-2), 1-135.

[7] Liu, B. (2012). Sentiment Analysis and Opinion Mining. Synthesis Lectures on Human Language Technologies, 5(1), 1-167.